WHAT THE BACKEND SHOULD DO

A practical implementation guide for reliable Falco reports, calculations, permissions and exports.

Backend objective

Make the backend the single source of truth. It must capture every reportable event, calculate metrics consistently, enforce role and branch access, return traceable report data, and generate exports from exactly the same authorized dataset.

Prepared 5 August 2026

Based on the uploaded Falco change request, the local web implementation, and the repository backend controller documentation.

1. Backend responsibilities

The backend owns reporting correctness

Frontend cards and charts should only present results. The backend must decide which records qualify, apply the date and branch scope, calculate the figures, enforce access, and provide the rows behind every total.

Responsibility	What the backend must do
Capture	Persist creator, branch, status changes, reasons, dates, payment allocations and other report source facts when events occur.
Validate	Reject invalid transitions, missing required reasons, unauthorized branch assignments and inconsistent financial allocations.
Calculate	Own canonical formulas for lead conversion, application turnaround, collection rate, PAR, NPL, aging and officer/branch performance.
Authorize	Apply report-type permissions and row-level branch/officer scope before the database query runs.
Explain	Return period, scope, data freshness, record counts and drill-down identifiers with report results.
Export	Create CSV, XLSX and PDF from the same query/service used by the JSON report response.
Audit	Record who created, approved, changed, reversed or exported sensitive financial information.

Rules that should never be delegated to the browser

- Creator identity: derive it from the authenticated token; do not trust a created_by value supplied by the browser.
- Branch scope and role permissions: reject unauthorized scope with HTTP 403.
- Financial calculations and aging: compute them using authoritative loan schedules, balances and verified payments.
- Export scope: never let a client-provided filename or filter bypass report permissions.

2. Frontend Reports menu to backend map

The sidebar now exposes the planned report destinations. The query parameter selects the frontend view; the backend endpoint supplies its authorized data. Until an endpoint is ready, the frontend should show a clear 'Backend report not available yet' state rather than invented or partial figures.

Frontend dropdown item	view value	Backend dependency
Reports Overview	default	Existing portfolio-summary, aging and timeseries, later combined through an overview service.
Lead Performance	leads-performance	GET /reports/leads-performance and /details; lead creator and lifecycle events.
Customer Demographics	customer-demographics	GET /reports/customer-demographics; DOB, gender, economic activity and geography.
Application Analytics	applications	GET /reports/applications; application status history and rejection reasons.
Expected Collections	expected-collections	GET /reports/expected-collections; schedules, verified payments and allocations.
Portfolio & Aging	portfolio-aging	GET /reports/portfolio-summary and /reports/aging with canonical risk formulas.
Disbursements	disbursements	GET /reports/disbursements with management fields and permissions.
Group Performance	groups-performance	GET /reports/groups-performance; attendance, collection and member exposure.
Financial Ledger	financial-ledger	GET /reports/financial-ledger; income, expense and non-client payment ledger.

Frontend/backend agreement

- The frontend sends only allowed filters and never calculates official totals from downloaded detail pages.
- The backend returns scope, period, summary, breakdowns, meta and traceability information in a consistent envelope.
- Every submenu supports the same date/branch/officer filter language where relevant.
- Exports receive the same filter set and report type, then the backend re-authorizes and recalculates; it must not reuse untrusted browser totals.

3. Data model and event history

Current-state fields are not enough for trend and turnaround reports. The backend needs immutable event history so it can reconstruct what happened and when.

Backend record	Required fields and behavior
leads	created_by_user_id, branch_id, gender, location/coordinates, current_status, created_at, converted_customer_id. Creator and branch are server assigned/validated.
lead_status_events	lead_id, from_status, to_status, occurred_at, actor_user_id, branch_id, reason_id and notes. Insert on every transition.
dropoff_reasons	Stable code, label, active flag and display order. Require a reason when a lead becomes lost or archived.
application_status_events	application_id, from/to status, actor, timestamp and rejection_reason_id. Needed for TAT, aging, rejection and abandonment.
loan schedules and balances	Installment due dates and components; principal, interest, fee and penalty balances; days past due and classification as of a specified date.
payment allocations	Payment and external reference; principal/interest/fee/penalty amounts; channel; verification, reconciliation and reversal audit.
group operations	Meeting date, attendance, expected collection, received collection, member balances, group risk and assigned officer.
branch/officer targets	Period, metric, target amount/count, branch, officer, approver and effective dates.
finance ledger	Other income, expense and non-client receipt entries with category, amount, branch, channel, approval and reversal history.

Historical migration

Backfill creator, timestamps and reasons only where reliable evidence exists. Keep unknown legacy values as 'Unknown / not recorded'; do not guess attribution or event dates.

4. Report services and API endpoints

Endpoint	Backend output
GET /reports/leads-performance	Lead totals, contact/application/final conversion rates, funnel, officer/branch ranking, stale leads, geography and drop-off reasons.
GET /reports/leads-performance/details	Paginated leads supporting a selected metric, including creator, branch, statuses/timestamps and conversion link.
GET /reports/customer-demographics	Counts by gender, approved age bands, economic activity, region and district.
GET /reports/applications	TAT, queue aging, rejection reasons, abandonment, counts and value by status, plus drill-down.
GET /reports/expected-collections	Due schedule rows and totals for today/week/month: expected, collected, shortfall, days overdue, branch and officer.
GET /reports/portfolio-summary	Canonical portfolio, PAR, NPL, provision, product and branch totals. Complete/retain documented behavior.
GET /reports/aging	Aging buckets and provision with loan-level traceability. Complete/retain documented behavior.
GET /reports/disbursements	Management detail including approval and disbursement actors/timestamps, branch, product and amount.
GET /reports/groups-performance	Attendance, repayment efficiency, group PAR, member concentration and officer capacity.
GET /reports/financial-ledger	Other income, expenses and non-client receipts by category, period, branch, channel and approval state.
GET /reports/{type}/export	CSV/XLSX/PDF binary using identical filters, permissions and calculations as the report JSON.

Common query filters

- from, to or as_of; branch_id; officer_id; product_id; status; page and page_size.
- Validate from <= to, accepted date format, maximum range, pagination limits and allowed granularity.

5. How each backend report request should run

Required request pipeline

Every report and export should pass through the same ordered pipeline. This prevents permission leaks, inconsistent formulas and differences between dashboard totals and downloaded files.

Step	Backend action	Failure behavior
1. Authenticate	Validate token/session and load user ID, role, branch and report permissions.	401 when authentication is missing or invalid.
2. Validate input	Parse report type, dates, filters, page size and export format. Normalize timezone/date boundaries.	422 with field-level validation details.
3. Resolve scope	Calculate effective branch/officer scope from the authenticated user. Ignore or reject broader client scope.	403 for forbidden report type or branch.
4. Query source data	Use indexed database queries over all qualifying records, not a capped frontend list.	Return a controlled 500 error and request/correlation ID; log technical details privately.
5. Calculate	Apply the single canonical report service for formulas, groupings and classifications.	Fail the report if required financial source data is internally inconsistent.
6. Reconcile	Check summary totals against grouped/detail totals where applicable and record data freshness.	Flag or block export when reconciliation fails.
7. Respond/export	Return JSON envelope or generate CSV/XLSX/PDF from the same result model.	404 for unsupported report; 422 for unsupported format.
8. Audit	Record sensitive report/export access with actor, scope, type, filters, timestamp and outcome.	Audit failure policy must be defined; sensitive export should fail closed if required by compliance.

Service structure

- Report controller: input validation, authorization and response formatting.
- Report query/service: database selection, grouping and canonical formulas.
- Report DTO/schema: stable JSON fields independent of database column names.
- Export renderer: CSV/XLSX/PDF created from the report DTO, never by re-querying with different logic.
- Audit/observability: correlation ID, duration, row count, applied scope and failure category without logging sensitive row content.

6. Canonical calculations

One formula, one owner

Each metric must have one backend definition shared by dashboards, detail views and exports. Store definitions in code and document numerator, denominator, event date, timezone, exclusions and rounding.

Metric	Recommended backend definition
Contact rate	Distinct leads that reached Contacted or a later qualifying stage / distinct leads created in the selected cohort x 100.
Application rate	Distinct cohort leads linked to an application / distinct leads created in the cohort x 100.
Final conversion	Distinct cohort leads linked to a disbursed loan / distinct leads created in the cohort x 100. Confirm this business definition before release.
Stale lead	Lead in New or Follow Up whose last meaningful activity is older than the configured threshold, initially 48 hours.
Application TAT	disbursed_at - approved_at. Also return median, percentiles and counts within/over the service target.
Collection rate	Verified collected amount for due items / expected scheduled amount for the same period x 100.
PAR ratio	Outstanding principal for qualifying loans in arrears / total outstanding principal of active portfolio x 100. Agree the PAR threshold, such as PAR30.
NPL ratio	Outstanding principal classified as non-performing / total outstanding principal x 100, following approved policy.
Officer productivity	Return separate volumes and quality measures; do not combine them into an unexplained score.

Historical timeseries warning

Do not repeat today's outstanding balance for past months. Build true month-end values from dated balance snapshots or reconstruct them from disbursement, schedule, payment, adjustment and reversal events.

7. Authorization, exports and reliability

Role and branch isolation

- Loan officer: assigned customers/portfolio within the permitted branch.
- Branch manager: assigned branch only.
- Super administrator: permitted global or selected-branch scope.
- Management disbursement and financial ledger reports: explicit report-type permission in addition to reports.view/reports.export.
- Enforce permissions before aggregation and export generation. Record sensitive exports in an audit log.

Response section	What it must contain
scope	Effective branch, officer, product and status filters after permission enforcement.
period	from, to, as_of, timezone and date inclusivity.
summary	Canonical metric values and qualifying record counts.
breakdowns	Grouped rows for charts/tables with stable codes and labels.
meta	generated_at, data freshness, page, page_size and total.
traceability	Detail endpoint or identifiers allowing each total to be reconciled to source records.

Operational reliability

- Aggregate in the database across the full authorized dataset. Do not calculate official totals from a frontend/server pagination cap such as 500 rows.
- Add indexes for report filters and joins: branch/date, officer/date, status/date, due_date, payment_date and foreign keys.
- Use consistent timezone handling and idempotent webhook/payment processing.
- Cache only when the cache key includes the full authorized scope and period; invalidate when source events change.

8. Production readiness and delivery controls

Concern	What must be implemented
Database performance	Explain/analyze critical queries; add composite indexes; avoid N+1 queries; use server-side aggregation and cursor/stable pagination for large detail sets.
Consistency	Use transactions for status transition plus event insert, payment plus allocations, and reversal plus audit. Prevent partial reporting state.
Idempotency	Deduplicate payment webhooks and retryable write commands using stable external/event IDs.
Freshness	Return data_fresh_as_of/generated_at. If snapshots are used, document refresh frequency and expose stale status.
Caching	Key by report type, complete effective scope, filters, formula version and freshness. Invalidate on qualifying source changes.
Observability	Structured logs and metrics for latency, failures, row counts, export size, reconciliation failures and permission denials.
Security	Parameterized queries, export formula-injection protection, safe filenames, output size limits, rate limits and no sensitive fields outside the report contract.
Testing	Unit formula tests; integration database tests; permission matrix tests; export parsing tests; scale tests beyond page limits; fixed timezone boundary fixtures.
Versioning	Version formula definitions and response schemas when a breaking calculation or field meaning changes.

Deployment checklist

- Apply schema migrations and indexes with a rollback plan.
- Run controlled historical backfill and publish counts of known versus unknown legacy values.
- Deploy endpoints behind report-specific feature flags where useful.
- Reconcile production-like sample totals with finance/operations before enabling exports.
- Enable submenu pages progressively only after their endpoint, permission tests and export tests pass.

9. Implementation order and acceptance tests

Phase	Backend delivery
1 - Definitions and audit	Approve formulas and scope rules; add creator attribution, lifecycle events, required reasons, allocation records and migrations.
2 - Lead reporting	Implement leads-performance summary/details and export. Reconcile creator, gender, branch, funnel, conversion, stale and drop-off results.
3 - Collections and risk	Implement expected collections, canonical portfolio/PAR/NPL/aging and accurate historical timeseries.
4 - Applications/disbursement	Implement TAT, queue aging, rejection analysis and management-only disbursement report/export.
5 - Broader reporting	Customer demographics, group performance, targets, income, expenses and non-client payments.

Backend acceptance checklist

Test	Pass condition
Attribution	New lead creator equals authenticated user in database, API and export; spoofing another creator fails.
Authorization	Cross-branch requests and unauthorized report types return 403 without leaking totals or rows.
Reconciliation	Summary equals detail-row aggregation and CSV/XLSX/PDF totals for identical filters.
Lifecycle	Every accepted status change creates one event with actor and timestamp; required reasons cannot be omitted.
Date behavior	Boundary dates, timezone and as_of calculations match the documented policy.
Finance	Reversals and duplicate webhooks do not double-count collections; allocations equal payment amount.
Scale	Results remain complete beyond 500 records and pagination returns stable, non-duplicated rows.
Legacy data	Unknown historic values are labelled and included according to the documented rule, never invented.

Definition of complete

The backend work is complete only when every report is permission-safe, traceable to detail records, consistent across screen and exports, accurate beyond pagination limits, and reproducible for a specified period or as-of date.